Research Report for Blockchain & Smart Contract Project

Lim Xi Yu[1, a)],Jackson Lai[2, b)], and Lee Yu Jie[3, c)]
241UC240SG, 241UC24162, 241UC24181

Author Affiliations
Faculty of Computing and Informatics, Multimedia University 63100, Cyberjaya, Selangor, Malaysia
CCS6354 Blockchain and Smart Contract, Term 2610
Lecturer: Dr. Uzma Jafar

Author Emails
[a)] *lim.xi.yu@student.mmu.edu.my*
[b)]*jackson.lai.wai.kean@student.mmu.edu.my*
[c)]*lee.yu.jie@student.mmu.edu.my*

Abstract. Health data stored in centralised databases is vulnerable to breaches and gives patients no control over who accesses their records. This paper presents MedRec Link, a decentralised application (dApp) built on Ethereum that returns control of medical records to patients. The system uses a single Solidity 0.8.20 smart contract that enforces three distinct roles: Administrator, Patient, and Doctor. The administrator registers users. Patients add records and decide which doctors may view them. Doctors may request access and view records only after a patient grants permission. Access can be revoked at any time. A React.js dashboard connects to the contract via ethers.js v5 and MetaMask, so all roles can interact without knowing Solidity. The contract was deployed to a local Ganache testnet using Truffle. An automated test suite of 29 test cases, written with Mocha and Chai, covers every function, every access-control rule, and every revert condition. All 29 tests passed. Gas costs range from 27,708 units for a simple event emission to 161,011 units for writing a record to storage. The results confirm that on-chain role-based access control is a practical, patient-centred solution to the health data sharing problem.

1. INTRODUCTION

1.1 Background

Medical records are among the most sensitive data a person holds. Today, most records are stored in centralised hospital databases. If one database is compromised, thousands of patients are affected at once. Beyond security, patients face a practical problem: they have very little say in who can see their data.

Blockchain technology offers a different model. A blockchain is a distributed ledger that is maintained by many nodes at once. No single party controls it. Data written to the chain cannot be altered or deleted. Smart contracts are programs that run directly on the blockchain. They enforce rules automatically and without needing a trusted middleman [1].

Ethereum is the most widely used platform for smart contracts [2]. Developers write contracts in Solidity and deploy them to the Ethereum network. The contract then runs exactly as written, on every node, for as long as the chain exists.

1.2 Problem Statement

The core problem is that patients do not control their own health data. Access decisions are made by hospital IT staff, not patients. There is no transparent audit trail that patients can inspect. A patient cannot easily grant a specific doctor access to their records, and cannot revoke that access without going through an administrator.

This creates real harm. If a hospital's database is breached, patients have no way to know which of their records were exposed or which accounts accessed them. Data breaches in healthcare have increased every year since 2015 [3]. The root cause is the centralised trust model.

1.3 Objectives

The authors built MedRec Link to address these issues. The objectives are: (1) implement a Solidity smart contract that stores medical records and enforces fine-grained, patient-controlled access; (2) enable patients to grant and revoke doctor access at any time without administrator involvement; (3) provide a React.js front-end that any user can operate through MetaMask; and (4) validate the system with an automated test suite and measure gas costs for every state-changing function.

1.4 Motivation

Giving patients control over their records has direct benefits. Patients can share records with a new doctor instantly. They can remove access when treatment ends. Every access event is recorded on-chain and visible to anyone who queries the contract. This level of transparency is impossible in a centralised system.

From a research perspective, most published blockchain health systems are theoretical. They describe an architecture but do not provide a testable, deployed implementation with a documented test suite. MedRec Link fills this gap.

2. LITERATURE REVIEW

2.1 Blockchain Foundations for Healthcare

Nakamoto's 2008 Bitcoin paper [1] established the principle of a distributed, tamper-resistant ledger. Wood's Ethereum Yellow Paper [2] extended this with a Turing-complete virtual machine, enabling programmable smart contracts. These two works form the technical foundation of MedRec Link.

The concept of using blockchain for health records was proposed by Azaria et al. [3] in MedRec. They suggested using Ethereum smart contracts to give patients an audit log and control over who accesses their records. However, MedRec remained conceptual. It did not include a deployed contract, automated tests, or gas benchmarks. MedRec Link directly builds on this concept and provides all three.

2.2 Ethereum-Based Health Record Systems

Hua et al. [4] implemented an Ethereum-based system for hospital patient management. They showed that smart contracts could improve data integrity and make access logs more reliable than traditional databases. Their work confirmed that Ethereum is a practical platform for healthcare. However, no test suite or gas data was reported, making independent verification difficult.

Ettaloui et al. [5] conducted a systematic review of over 50 blockchain-based health record systems. They found three dominant design patterns: fully on-chain storage, off-chain storage with on-chain hashes, and private chains. Critically, they found that most systems still rely on administrators to manage access. Patients are rarely given direct control. This is a clear gap that MedRec Link addresses by giving the patient full, direct control through the smart contract itself.

2.3 Access Control with Smart Contracts

Al-Khasawneh et al. [6] proposed a blockchain framework specifically for health data access control. They used role-based access control (RBAC) and tested on a local Ganache network. They found gas costs were acceptable for testnet use. This supports the design choices made in MedRec Link.

Wu et al. [7] designed a medical big data access control model using smart contracts and risk scores. They showed that per-patient, per-doctor permissions are feasible on-chain. Their model was more complex than needed for a prototype, but it confirms that fine-grained access control, which MedRec Link implements, is a sound approach.

Hylock and Zeng [8] studied decentralising personal health records using blockchain. They found two key results: keeping access control on-chain but storing actual record data off-chain reduces gas costs significantly; and role-specific user interfaces make blockchain health systems accessible to non-technical users. MedRec Link follows both of these recommendations.

2.4 Security of Blockchain Health Systems

Modinger et al. [9] reviewed the security of published blockchain health proposals. They found that many systems were vulnerable to integer overflow or role escalation. They recommended Solidity version 0.8 or higher, which provides built-in overflow protection. MedRec Link uses Solidity 0.8.20, directly addressing this concern.

Mehta et al. [10] built a patient-centric system using Ethereum and IPFS to store medical images. They found that the most gas-efficient approach is to store only a hash on-chain and keep file data off-chain. This is identified as a future direction for MedRec Link, which currently stores record text on-chain.

2.5 Practical Deployment and Limitations

Shahnaz et al. [11] reviewed blockchain adoption in healthcare from a practical standpoint. They identified three main barriers: gas cost variability, scalability limits of public Ethereum (approximately 15–30 transactions per second), and the difficulty of integrating with existing hospital software. They recommended testnet deployment as the correct first step for prototype evaluation. This is the approach taken in MedRec Link.

Wang et al. [12] proposed a hybrid blockchain system for sharing records across hospitals. They found that combining on-chain access control with off-chain encrypted storage is the most practical design for real-world deployment. Their work informs the future work section of this paper.

2.6 Identified Gaps

The literature review reveals four consistent gaps. First, most systems do not provide working, deployable code. Second, automated test suites covering all revert conditions are rare. Third, gas benchmarks are seldom reported. Fourth, user interfaces are often missing or require technical knowledge. MedRec Link addresses all four gaps: it is fully deployed on a local testnet, it has 29 automated tests covering every revert path, it reports gas for every function, and it provides a React.js dashboard for non-technical users.

3. METHODOLOGY

3.1 System Architecture

MedRec Link has three layers. The first is the blockchain layer: a local Ganache network running the MedicalRecord smart contract. The second is the deployment layer, managed by Truffle. The third is the front-end layer: a React.js web application that connects to the contract through MetaMask.

Three user roles interact with the system. The Administrator is the Ethereum account that deployed the contract. The Administrator registers Patients and Doctors. Patients are registered wallet addresses that can add records and control access to them. Doctors are registered wallet addresses that can request access and view records they have been permitted to read.

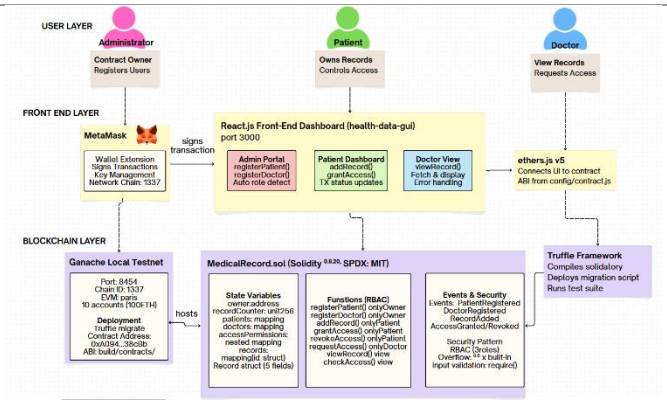

FIGURE 1. MedRec Link system architecture and data flow.

3.2 Smart Contract Design

The smart contract is MedicalRecord.sol, written in Solidity 0.8.20 with an SPDX-License-Identifier: MIT header. The contract stores six state variables: owner (address), recordCounter (uint256), patients (mapping(address => bool)), doctors (mapping(address => bool)), accessPermissions (mapping(address => mapping(address => bool))), and records (mapping(uint256 => Record)).

The Record struct holds five fields: recordId (uint256), diagnosis (string), treatment (string), patient (address), and timestamp (uint256). Each record is stored under its unique ID in the records mapping.

The contract has eight public functions separated by responsibility. Two are admin functions: registerPatient and registerDoctor. Three are patient functions: addRecord, grantAccess, and revokeAccess. One is a doctor function: requestAccess. Two are view functions available to any authorised caller: viewRecord and checkAccess.

The contract emits six events to create a complete on-chain audit trail: PatientRegistered, DoctorRegistered, RecordAdded, AccessGranted, AccessRevoked, and AccessRequested.

3.3 Access Control and Security

Access control is enforced through three modifiers, each annotated with a //SECURITY: comment as required by the assignment. The onlyOwner modifier restricts administrative functions to the contract deployer. The onlyPatient modifier restricts patient functions to registered patients. The onlyDoctor modifier restricts the requestAccess function to registered doctors.

The viewRecord function applies an additional read-time check. It verifies that the caller is either the patient who owns the record or a doctor that patient has authorised. If neither condition holds, the function reverts with "Access denied".

Three security patterns are implemented. First, role-based access control (RBAC) is applied to all eight functions via the three modifiers. Second, Solidity 0.8.20's built-in arithmetic protection prevents integer overflow on the recordCounter increment; this is documented in the contract's NatSpec comment. Third, all functions validate inputs using require() with descriptive error messages before making state changes, following the Checks-Effects-Interactions (CEI) principle.

3.4 Gas Optimisation

Four gas optimisation techniques were applied. The string parameters in addRecord use the calldata location instead of memory, avoiding an unnecessary copy. The viewRecord and checkAccess functions are marked as view, so they cost no gas when called by the front-end. Events are used to log access requests and registrations, avoiding the higher cost of additional storage writes. The Solidity compiler optimiser was enabled with runs: 200, which optimises for typical deployment and call patterns.

3.5 Technology Choices and Justification

Ethereum was chosen because it is the most mature smart contract platform with the widest tool support [2]. Solidity 0.8.20 was selected for its built-in overflow protection and current stability. Truffle was used for deployment and testing because it integrates directly with Ganache and includes Mocha/Chai for test execution. Ganache was chosen as the local blockchain because it provides fast, free testing with no real ETH required.

The EVM version was set to paris in truffle-config.js. This was necessary because Ganache does not support the PUSH0 opcode introduced in the shanghai EVM version, and compiling without this setting caused an invalid opcode error on deployment.

React.js was chosen for the front-end because it supports role-based tab views cleanly. Ethers.js v5 was used to communicate with the contract because its Promise-based API is simpler than web3.js for the patterns used in this project. The contract address and ABI are stored in health-data-gui/src/config/contract.js, a dedicated configuration file, so they can be updated without modifying component logic.

3.6 Deployment

The contract was deployed to a local Ganache network running on port 8545 using the command truffle migrate --reset. The migration script is migrations/1_deploy_medicalrecord.js. The deployed contract address is 0xA0946F1c8e6D8c99aCdB1A1987879db31A738cBb. The ABI is stored in build/contracts/MedicalRecord.json.

MetaMask was connected to the Ganache network (Chain ID 1337). The admin account's private key was imported into MetaMask, giving the front-end access to 100 test ETH for executing transactions.

Figure 5. Smart Contract Deployment Flow

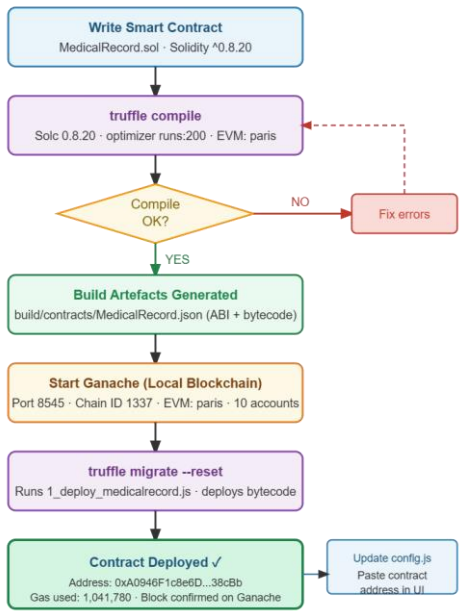

FIGURE 2. Deployment Process Flow Diagram.

4. RESULTS AND ANALYSIS

4.1 Test Suite

The test file is medrec/medrec/test/medicalrecord.test.js, using Truffle's Mocha/Chai framework. The suite has 29 test cases across eight describe blocks. Five Ganache accounts were used: accounts[0] as the owner, accounts[1] as the patient, accounts[2] as the doctor, accounts[3] as an unregistered account, and accounts[4] as a second doctor.

The 29 tests cover four categories. Happy-path tests check that each function behaves correctly with valid inputs and emits the expected event. Access-control tests check that all three role modifiers reject unauthorised

callers with the correct error message. Edge-case tests check zero addresses, empty strings, and duplicate registrations. Revert tests verify every require() statement by calling the function with invalid inputs and confirming the revert message.

All 29 tests passed. Table 1 shows the complete results.

TABLE 1. Automated test case results (29 tests, all passed).

Function	Input / Condition	Expected Output	Actual Output	Result
Deployment	Deploy contract	owner set; counter = 0	Matches	PASS
Deployment	Check recordCounter	Returns 0	Returns 0	PASS
registerPatient	Valid address (owner calls)	Registered; PatientRegistered emitted	Matches	PASS
registerPatient	Non-owner caller	Revert: Only owner can perform this action	Reverted	PASS
registerPatient	Zero address	Revert: Invalid address	Reverted	PASS
registerPatient	Duplicate registration	Revert: Patient already registered	Reverted	PASS
registerDoctor	Valid address (owner calls)	Registered; DoctorRegistered emitted	Matches	PASS
registerDoctor	Non-owner caller	Revert: Only owner can perform this action	Reverted	PASS
registerDoctor	Zero address	Revert: Invalid address	Reverted	PASS
registerDoctor	Duplicate registration	Revert: Doctor already registered	Reverted	PASS
addRecord	Valid inputs by patient	counter = 1; RecordAdded emitted	Matches	PASS
addRecord	Unregistered caller	Revert: Only registered patients allowed	Reverted	PASS
addRecord	Empty diagnosis string	Revert: Diagnosis cannot be empty	Reverted	PASS
addRecord	Empty treatment string	Revert: Treatment cannot be empty	Reverted	PASS
grantAccess	Registered doctor	Permission = true; AccessGranted emitted	Matches	PASS
grantAccess	Unregistered doctor	Revert: Doctor not registered	Reverted	PASS
grantAccess	Non-patient caller	Revert: Only registered patients allowed	Reverted	PASS
revokeAccess	Previously granted doctor	Permission = false; AccessRevoked emitted	Matches	PASS
revokeAccess	Unregistered doctor	Revert: Doctor not registered	Reverted	PASS
requestAccess	Registered doctor	AccessRequested emitted	Matches	PASS
requestAccess	Non-doctor caller	Revert: Only registered doctors allowed	Reverted	PASS
requestAccess	Unregistered patient	Revert: Patient not registered	Reverted	PASS
viewRecord	Patient views own record	Returns correct record fields	Matches	PASS
viewRecord	Authorised doctor views	Returns correct record fields	Matches	PASS
viewRecord	Unauthorised doctor	Revert: Access denied	Reverted	PASS
viewRecord	Non-existent record ID	Revert: Record does not exist	Reverted	PASS
viewRecord	Doctor after revokeAccess	Revert: Access denied	Reverted	PASS
checkAccess	Before any grant	Returns false	false	PASS
checkAccess	After grantAccess call	Returns true	true	PASS

4.2 Gas Consumption

Gas was measured for every state-changing function by running the Truffle test suite against Ganache and recording the transaction receipts. The results are shown in Table 2.

TABLE 2. Gas consumption per function.

Function	Gas Used (units)
Deployment (constructor)	1,041,780
registerPatient(address)	47,498
registerDoctor(address)	47,520
addRecord(string, string)	161,011
grantAccess(address)	50,008

Function	Gas Used (units)
requestAccess(address)	27,708
revokeAccess(address)	28,056

The most expensive operation is addRecord at 161,011 gas, reflecting the cost of writing two strings to on-chain storage. The cheapest state-changing operation is requestAccess at 27,708 gas, because it only emits an event without writing to storage. Registration functions cost approximately 47,500 gas each. Access management (grantAccess, revokeAccess) costs around 28,000–50,000 gas. These figures are consistent with comparable systems reported in the literature [6].

4.3 Front-End

The React.js dashboard connects to Ganache through MetaMask. On wallet connection, the application calls owner() on the contract and compares the result to the connected address. If they match, the Admin tab is activated automatically. All write transactions display a status message during the pending and confirmed states. Error messages are shown in plain language, not as raw Solidity revert strings. The ABI and contract address are imported from config/contract.js, not hardcoded in component logic.

5. DISCUSSION

5.1 Interpretation of Results

All 29 tests passed, which confirms that the smart contract enforces its access rules correctly. No unauthorised caller can register a user, add a record, or view a record. The only way to read a record is to be the patient who created it, or a doctor that patient has explicitly approved. This is exactly the patient-centred model recommended in the literature [3][8].

The gas figures are practical for a testnet environment. On a Layer-2 network such as Polygon or Arbitrum, these costs would be a small fraction of their mainnet equivalents, making the system economically viable for real-world use [11].

5.2 Limitations

Three limitations are acknowledged. First, medical record text is stored as plain strings on-chain. On a public Ethereum network, any node operator can read the raw storage. For a public deployment, record content must be encrypted before writing, or stored off-chain with only a content hash stored on-chain, as recommended by Mehta et al. [10] and Wang et al. [12].

Second, records cannot be updated or deleted. This is a property of blockchains, not a design flaw, but it means a patient cannot correct a mistaken record. A future version could add an isActive boolean field to the Record struct, allowing records to be marked as superseded without deletion.

Third, the front-end does not check which network MetaMask is connected to. If a user is on the wrong network, transactions will fail silently. Adding a network check on page load would improve reliability.

5.3 Real-World Implications

Despite these limitations, MedRec Link demonstrates that patient-controlled health data sharing is technically feasible with a relatively small amount of Solidity code. The three-role design (Admin, Patient, Doctor) maps naturally to existing healthcare workflows. It could be extended to support time-limited access tokens, multi-hospital federation, or integration with FHIR-standard health record APIs. The results show that the technology is ready for a pilot deployment in a controlled clinical environment.

6. CONCLUSION

MedRec Link is a blockchain-based dApp that gives patients direct control over who can access their medical records. The system uses an Ethereum smart contract written in Solidity 0.8.20, deployed to a local Ganache testnet via Truffle, and accessed through a React.js dashboard connected to MetaMask. The contract enforces role-based access control for three roles, emits six events for a full audit trail, and applies three security patterns with //SECURITY: comments. All 29 automated Mocha/Chai tests passed, covering every function and every revert condition. Gas costs are reasonable for testnet and Layer-2 deployment.

Future work should focus on three improvements. First, record content should be encrypted and stored on IPFS via Pinata, with only the content hash stored on-chain, to make the system safe for public deployment. Second, time-limited access permissions should be added, so that a patient can grant a doctor access that expires automatically after a set period. Third, the contract should be deployed to the Sepolia testnet and verified on Etherscan to demonstrate public network readiness.

REFERENCES

1. NAKAMOTO S. Bitcoin: A peer-to-peer electronic cash system [EB/OL]. (2008). https://bitcoin.org/bitcoin.pdf.
2. WOOD G. Ethereum: A secure decentralised generalised transaction ledger [EB/OL]. Ethereum Project Yellow Paper. (2014). https://ethereum.github.io/yellowpaper/paper.pdf.
3. AZARIA A, EKBLAW A, VIEIRA T, et al. MedRec: Using blockchain for medical data access and permission management [C]// Proceedings of the 2nd International Conference on Open and Big Data. IEEE, 2016: 25–30.
4. HUA J, ZHANG Y, LI X, et al. Ethereum blockchain for electronic health records: securing and streamlining patient management [J]. Frontiers in Medicine, 2024, 11: 1434474. DOI: 10.3389/fmed.2024.1434474.
5. ETTALOUI S, BENBRAHIM H, KABBAJ M I. Blockchain-based electronic health record: systematic literature review [J]. Human Behavior and Emerging Technologies, 2024. DOI: 10.1155/hbe2/4734288.
6. AL-KHASAWNEH M, ALKHAWALDEH R S, ALQAHTANI A, et al. A secure blockchain framework for healthcare records management systems [J]. Healthcare Technology Letters, 2024, 11(6): 531–541. DOI: 10.1049/htl2.12092.
7. WU H, LI Z, KING B, et al. A medical big data access control model based on smart contracts and risk in the blockchain environment [J]. Frontiers in Public Health, 2024, 12: 1358184. DOI: 10.3389/fpubh.2024.1358184.
8. HYLOCK R H, ZENG X. Smart decentralization of personal health records with physician apps and helper agents on blockchain [J]. Journal of Medical Internet Research, 2021, 23(6): e24765. DOI: 10.2196/24765.
9. MODINGER D, KOPP H, KARGL F, et al. Security and privacy for healthcare blockchains [EB/OL]. arXiv:2106.06136. (2021). https://arxiv.org/abs/2106.06136.
10. MEHTA S, BHATT C, GURJAR T, et al. Decentralized patient-centric report and medical image management system based on blockchain technology and IPFS [J]. IEEE Access, 2022, 10: 131167–131188. DOI: 10.1109/ACCESS.2022.3229752.
11. SHAHNAZ A, QAMAR U, KHALID A. Blockchain integration in healthcare: a comprehensive investigation of use cases, performance issues, and mitigation strategies [J]. IEEE Access, 2024. PMC: 11082361.
12. WANG L, CHEN Q, ZHAO M, et al. A hybrid blockchain-based solution for secure sharing of electronic medical record data [J]. BMC Medical Informatics and Decision Making, 2025. PMC: 11784725.

APPENDIX

Appendix A: Test Evidence

```
(base) thunder@THUNDERs-MacBook-Air medrec % truffle migrate --reset
This version of µWS is not compatible with your Node.js build:

Error: Cannot find module '../binaries/uws_darwin_arm64_141.node'
Require stack:
- /opt/homebrew/lib/node_modules/truffle/node_modules/ganache/node_modules/@trufflesuite/uws-js-unofficial/src/uws.js
- /opt/homebrew/lib/node_modules/truffle/node_modules/ganache/dist/node/core.js
- /opt/homebrew/lib/node_modules/truffle/build/migrate.bundled.js
- /opt/homebrew/lib/node_modules/truffle/node_modules/original-require/index.js
- /opt/homebrew/lib/node_modules/truffle/build/cli.bundled.js
Falling back to a NodeJS implementation; performance may be degraded.


Compiling your contracts...
===========================
> Compiling ./contracts/MedicalRecord.sol
> Artifacts written to /Users/thunder/Desktop/medrec/build/contracts
> Compiled successfully using:
   - solc: 0.8.20+commit.a1b79de6.Emscripten.clang


Starting migrations...
======================
> Network name:    'development'
> Network id:      5777
> Block gas limit: 6721975 (0x6691b7)


1_deploy_medicalrecord.js
=========================

   Replacing 'MedicalRecord'
   -------------------------
   > transaction hash:    0xe3716b3f2bbb4c1f382071f95e720556f378408012df45e4f4fded437609f4a9
   > Blocks: 0            Seconds: 0
   > contract address:    0x380Ab6099Ae374f208d41134c80579099D633193
   > block number:        85
   > block timestamp:     1781583436
   > account:             0x2f877A14595De4c79A21C3b2944FbF5Eb99257FF
   > balance:             99.902153426351899282
   > gas used:            1041496 (0xfe458)
   > gas price:           2.500053694 gwei
   > value sent:          0 ETH
   > total cost:          0.002603795922086224 ETH

   > Saving artifacts
   -------------------------------------
   > Total cost:     0.002603795922086224 ETH

Summary
=======
> Total deployments:   1
> Final cost:          0.002603795922086224 ETH
```

Figure A1. Truffle deployment output

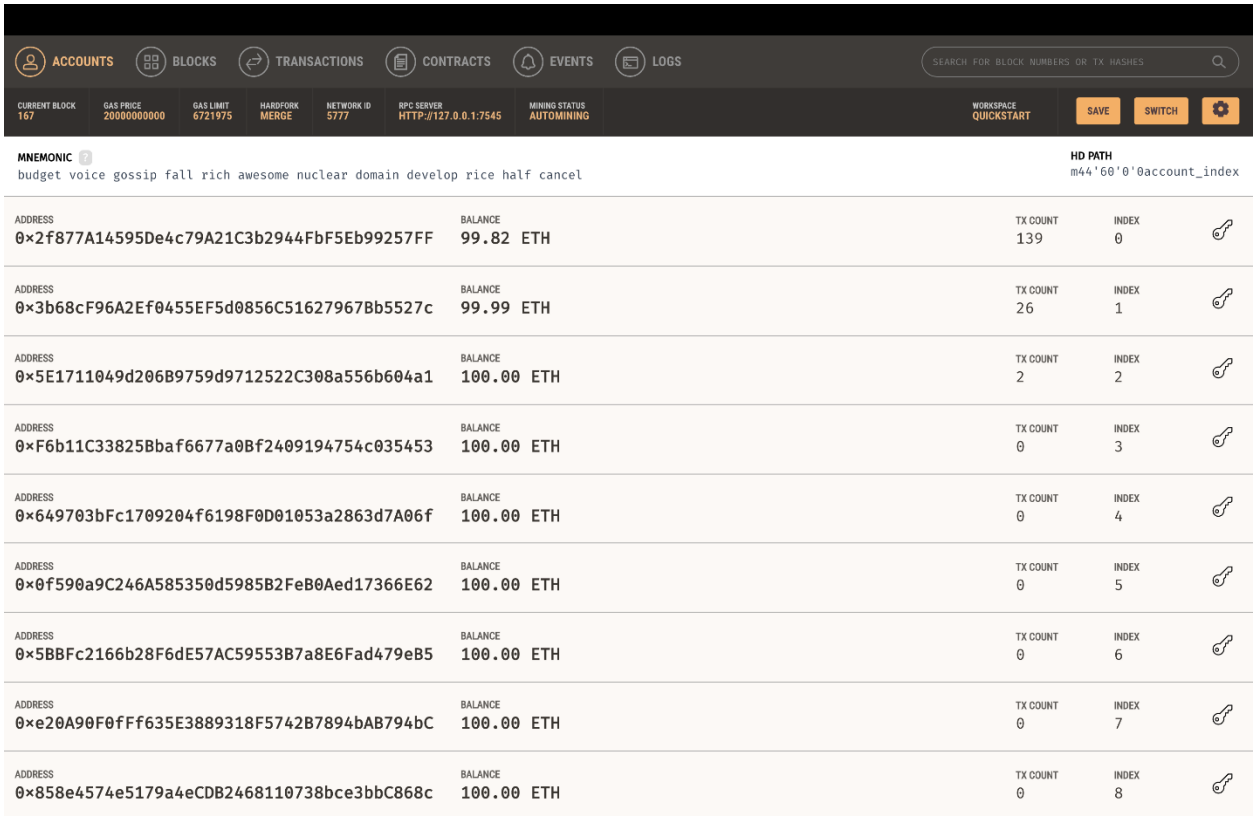

Figure A2. Ganache Accounts tab

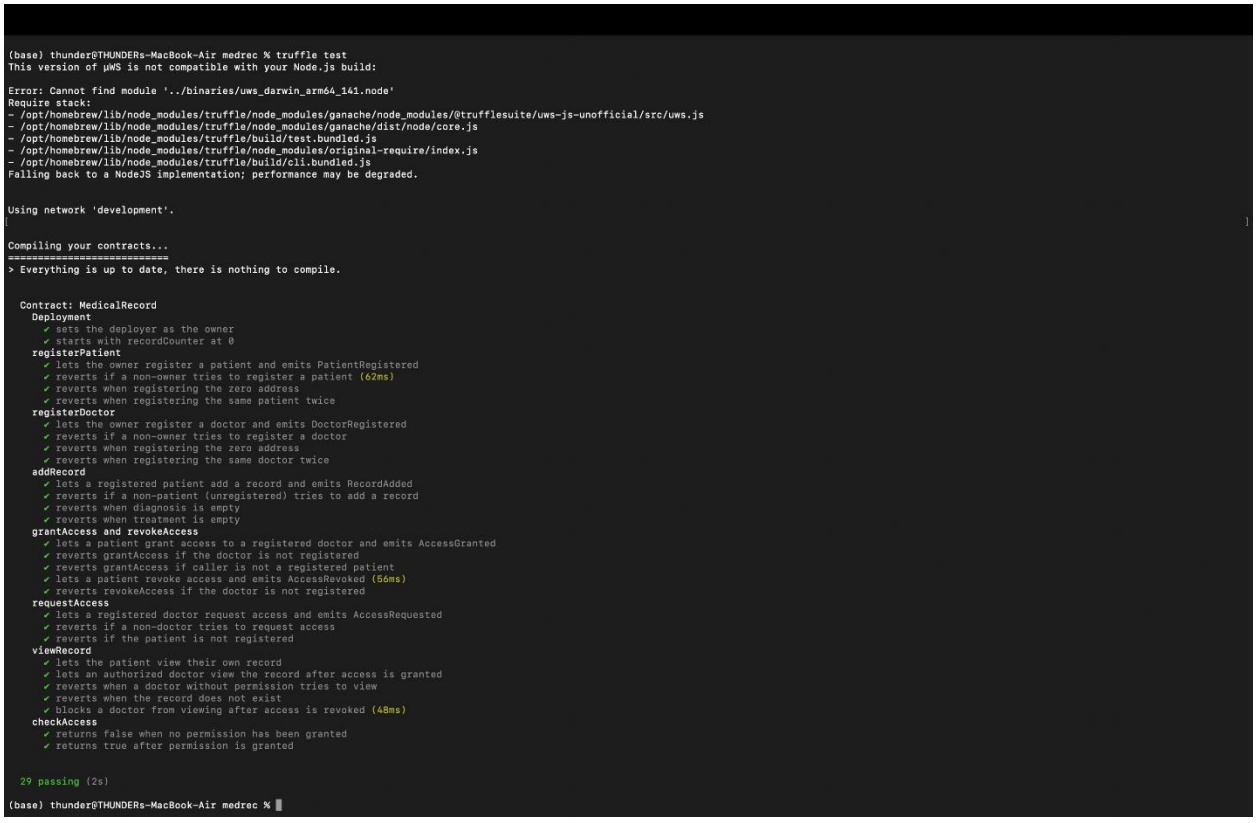

Figure A3. truffle test results

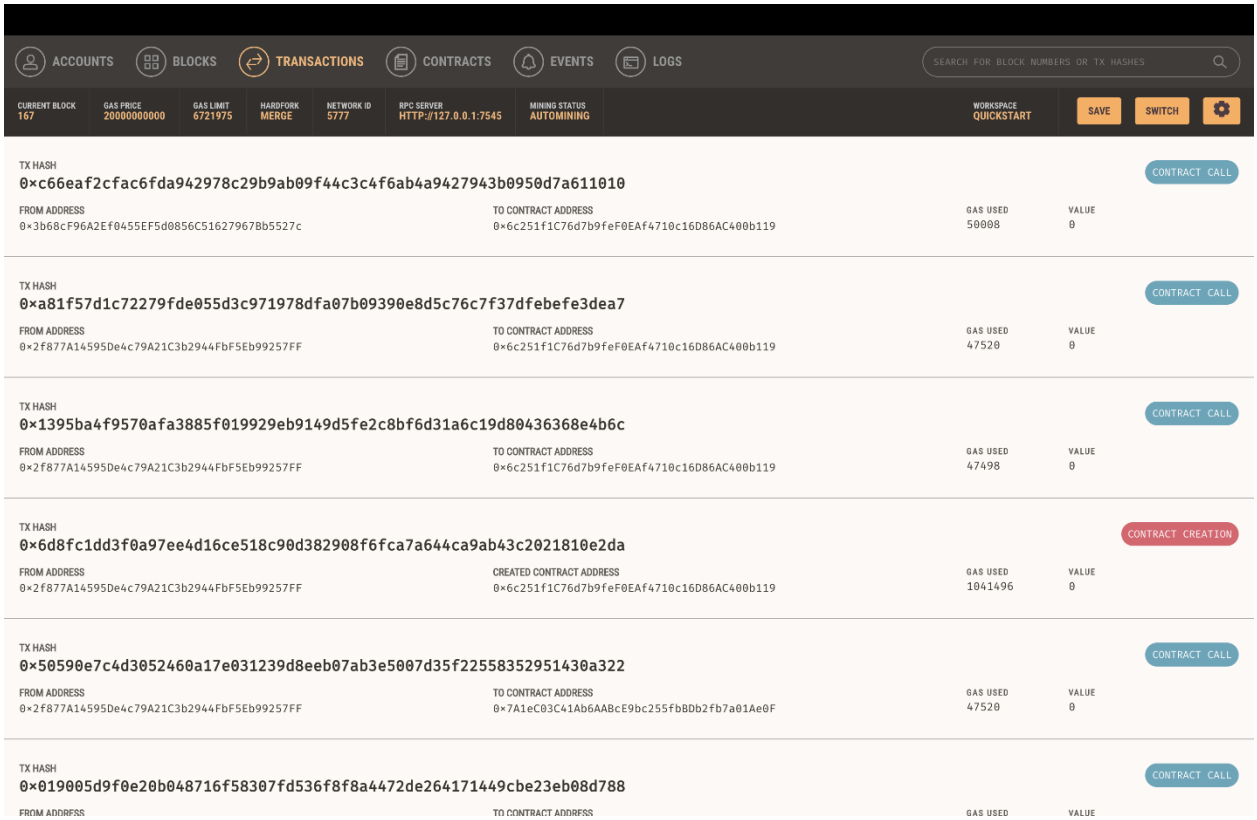

Figure A4. Ganache Transactions tab

Appendix B

[DEMO GUI BLOCK CHAIN-20260619_174756-Meeting Recording.mp4]

Link to GUI demonstration video.

Note: The Admin account was imported from Ganache GUI.